

Unidad 2

Relación lineal entre variables, función de costo (MSE), método del gradiente y ajuste de parámetros. Funciones convexas y no convexas. Descenso del gradiente, tasa de aprendizaje, épocas. Variantes del tipo de gradiente. Descomposición del error en bias y varianza. Regularización: Ridge, Lasso y Elastic Net como estrategias para reducir sobreajuste y manejar multicolinealidad en datos altamente correlacionados. Aplicaciones Espaciales

Regresión

Relación lineal entre variables

Hablamos de una relación lineal cuando el cambio en una variable (independiente, x) produce un cambio proporcional y constante en otra variable (dependiente, y).

En términos visuales, si trazamos estos puntos en un gráfico de dispersión, notarás que tienden a alinearse siguiendo una trayectoria recta.

En el álgebra lineal aplicada al aprendizaje automático, la relación lineal se expresa así:

$$\mathbf{y} = \mathbf{X} \cdot \mathbf{w} + \epsilon$$

Donde:

- y : Es el **vector de etiquetas** (los valores reales que queremos predecir).
- X : Es la **matriz de características** (los datos de entrada).
- w : Es el **vector de pesos** o parámetros (incluye la **pendiente m** y el **bias b**).
- ϵ (**epsilon**): Representa el error o ruido que el modelo no puede explicar.

Notación matricial de una relación lineal

$$y_1 = \beta_0 + \beta_1 x_1 + e_1$$

$$y_2 = \beta_0 + \beta_1 x_2 + e_2$$

$$y_3 = \beta_0 + \beta_1 x_3 + e_3$$

$$\vdots \qquad \qquad \vdots \qquad \qquad \vdots$$
$$y_n = \beta_0 + \beta_1 x_n + e_n$$



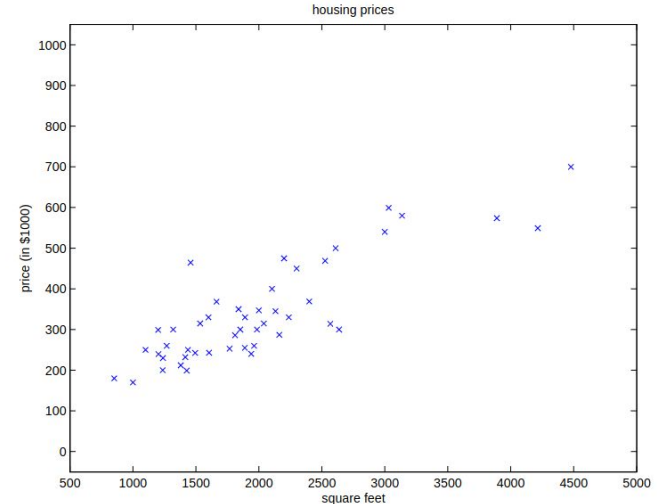
$$\begin{pmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_n \end{pmatrix} = \begin{pmatrix} \beta_0 + \beta_1 x_1 \\ \beta_0 + \beta_1 x_2 \\ \beta_0 + \beta_1 x_3 \\ \vdots \\ \beta_0 + \beta_1 x_n \end{pmatrix} + \begin{pmatrix} e_1 \\ e_2 \\ e_3 \\ \vdots \\ e_n \end{pmatrix} \quad \longrightarrow \quad \begin{pmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_n \end{pmatrix} = \begin{pmatrix} 1 & x_1 \\ 1 & x_2 \\ 1 & x_3 \\ \vdots & \vdots \\ 1 & x_n \end{pmatrix} * \begin{pmatrix} \beta_0 \\ \beta_1 \end{pmatrix} + \begin{pmatrix} e_1 \\ e_2 \\ e_3 \\ \vdots \\ e_n \end{pmatrix}$$

$$y = X.w + \text{epsilon}$$

Regresión lineal

Supongamos que pretendemos modelar el precio de una propiedad teniendo en cuenta los metros cuadrados de esa propiedad.

Living area (feet ²)	Price (1000\$s)
2104	400
1600	330
2400	369
1416	232
3000	540
⋮	⋮



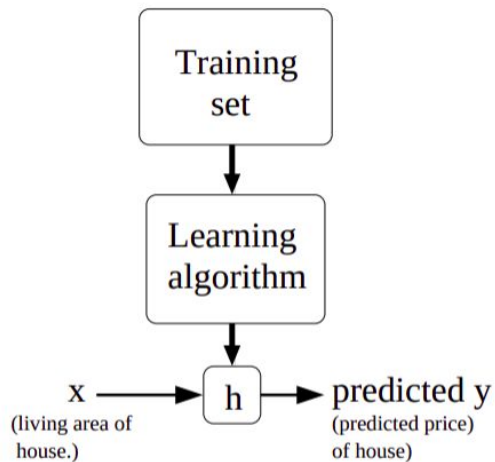
Regresión lineal

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

$$h(x) = \sum_{i=0}^n \theta_i x_i = \theta^T x,$$

$$J(\theta) = \frac{1}{2} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2.$$

Función de
costo



El parámetro
theta se
actualiza
según

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta).$$

tasa de aprendizaje
(α o η)

Función de costo

En el contexto del aprendizaje supervisado, la **Función de Costo** (también llamada *Cost Function* o *Objective Function*) es una función matemática $J\theta$ que mapea los parámetros de un algoritmo a un número escalar que representa el "costo" o "penalización" asociado a esos parámetros.

Su objetivo es promediar el error de la **Función de Pérdida** (*Loss Function*) a lo largo de todo el conjunto de entrenamiento. Mientras que la pérdida mide el error en un solo ejemplo, el costo mide el rendimiento global del modelo.

Para la regresión lineal, la función de costo por excelencia es el **Mean Squared Error (MSE)**. Matemáticamente, se define como:

Hypothesis: $h_{\theta}(x) = \theta_0 + \theta_1 x$

Parameters: θ_0, θ_1

Cost Function: $J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$

Goal: $\underset{\theta_0, \theta_1}{\text{minimize}} J(\theta_0, \theta_1)$

Convexidad

En la regresión lineal, el MSE genera una superficie de error **convexa** (con forma de cuenco o parábola multidimensional). Esto es fundamental porque garantiza la existencia de un **mínimo global** único. No hay "trampas" o mínimos locales donde el modelo pueda quedar atrapado.

Penalización Cuadrática

Al elevar la diferencia al cuadrado $\hat{Y} - Y^2$:

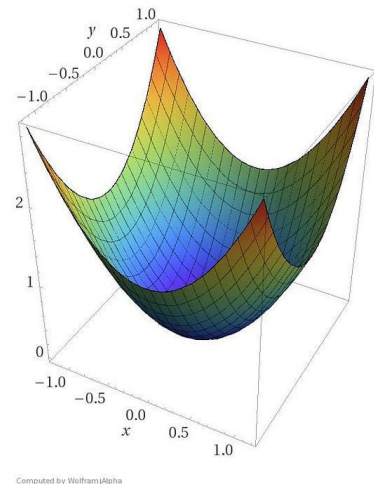
- **Magnifica los errores grandes:** Un error de 10 unidades se convierte en una penalización de 100, mientras que uno de 2 se convierte en 4. Esto obliga al modelo a priorizar la corrección de los *outliers* o grandes desviaciones.
- **Asegura valores positivos:** No importa si la predicción es por exceso o por defecto; el costo siempre será mayor o igual a cero.

Diferenciabilidad

Para que podamos optimizar el modelo mediante el **Método del Descenso del Gradiente**, la función de costo debe ser derivable. El MSE es suave y continuo, lo que permite calcular su pendiente en cualquier punto para saber hacia dónde "bajar" el error.

Una función es **convexa** si, geométricamente, cualquier segmento de recta trazado entre dos puntos de su gráfica queda por encima (o sobre) la curva. En términos simples, tiene forma de "U" o de cuenco.

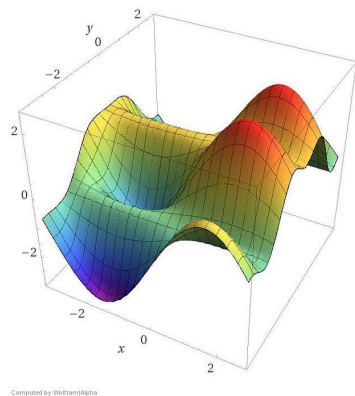
- **Punto clave:** Solo tienen un **mínimo global**. No hay mínimos locales.
- **Ejemplo en ML:** La Regresión Lineal con la función de costo **MSE** (Error Cuadrático Medio) y la Regresión Logística.
- **Ventaja:** El método del gradiente siempre llegará al mismo punto óptimo, sin importar desde dónde empieces.



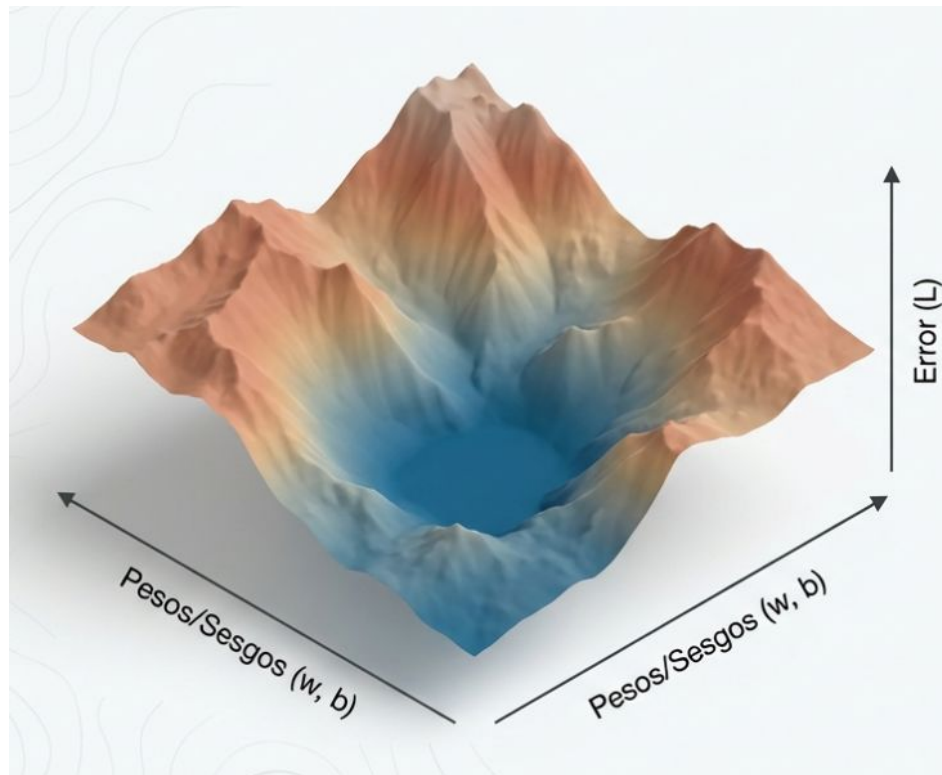
Funciones no convexas

Las funciones **no convexas** tienen una topografía mucho más accidentada. Pueden tener múltiples "valles" (mínimos locales), "picos" (máximos) y zonas llanas (puntos de silla).

- **Punto clave:** El algoritmo puede quedar atrapado en un **mínimo local**, creyendo que ha llegado al mejor resultado cuando en realidad existe un **mínimo global** mucho más bajo en otra parte de la superficie.
- **Ejemplo en ML:** Las **Redes Neuronales Profundas (Deep Learning)**. Debido a la composición de múltiples capas y funciones de activación no lineales, la superficie de error es extremadamente compleja.
- **Desafío:** El resultado final puede depender de la "suerte" (cómo inicializar los pesos al azar al principio).



Funciones no convexas



El Objetivo:

Entrenar un modelo es un problema puro de minimización matemática.

Eje X/Y (Pesos/Sesgos):

Los parámetros configurables de la red neuronal.

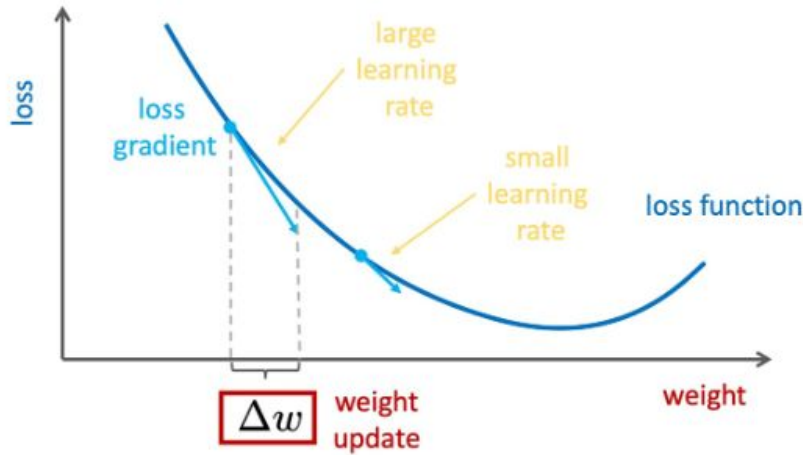
Eje Z (Error/Loss):

La diferencia entre la predicción y la realidad.

La Meta:

Encontrar la combinación exacta de pesos que lleve el error al punto más profundo del valle.

Tasa de aprendizaje



Cuando calculamos el gradiente, este nos indica la *dirección* hacia la cual la función de costo crece más rápido. Para minimizar el error, nos movemos en dirección opuesta. La tasa de aprendizaje determina qué tan grande será ese movimiento.

- Si η es **muy pequeño**: El algoritmo avanza con pasos de hormiga. El entrenamiento será extremadamente lento y podría quedarse estancado antes de llegar al mínimo.
- Si η es **muy grande**: El algoritmo da saltos de gigante. Esto puede causar que "rebote" de un lado a otro del valle, superando el mínimo sin tocarlo, e incluso puede hacer que el error no converja (aumente al infinito).

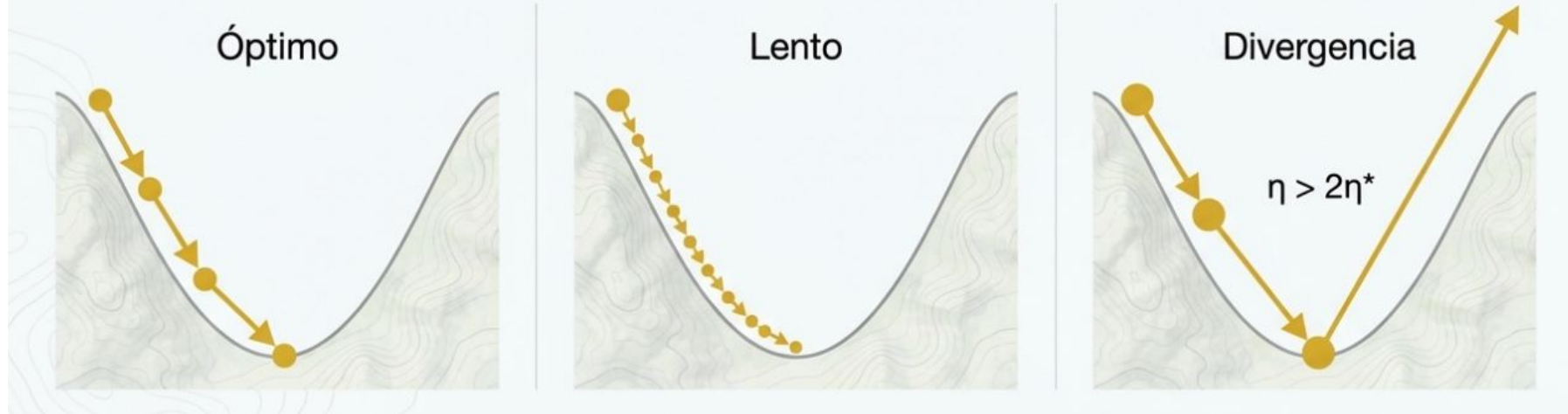
$$\boxed{w} := \boxed{w} - \underbrace{\boxed{\eta} \boxed{\nabla Q_i(w)}}_{\text{gradient descent}}$$

new weight previous weight learning rate gradient of the loss function

Concepto: La Tasa de Aprendizaje (η) define el tamaño de cada paso.

Riesgo: Demasiado pequeña = convergencia interminable.

Demasiado grande = oscilaciones destructivas y divergencia total.



Tasa de aprendizaje y épocas

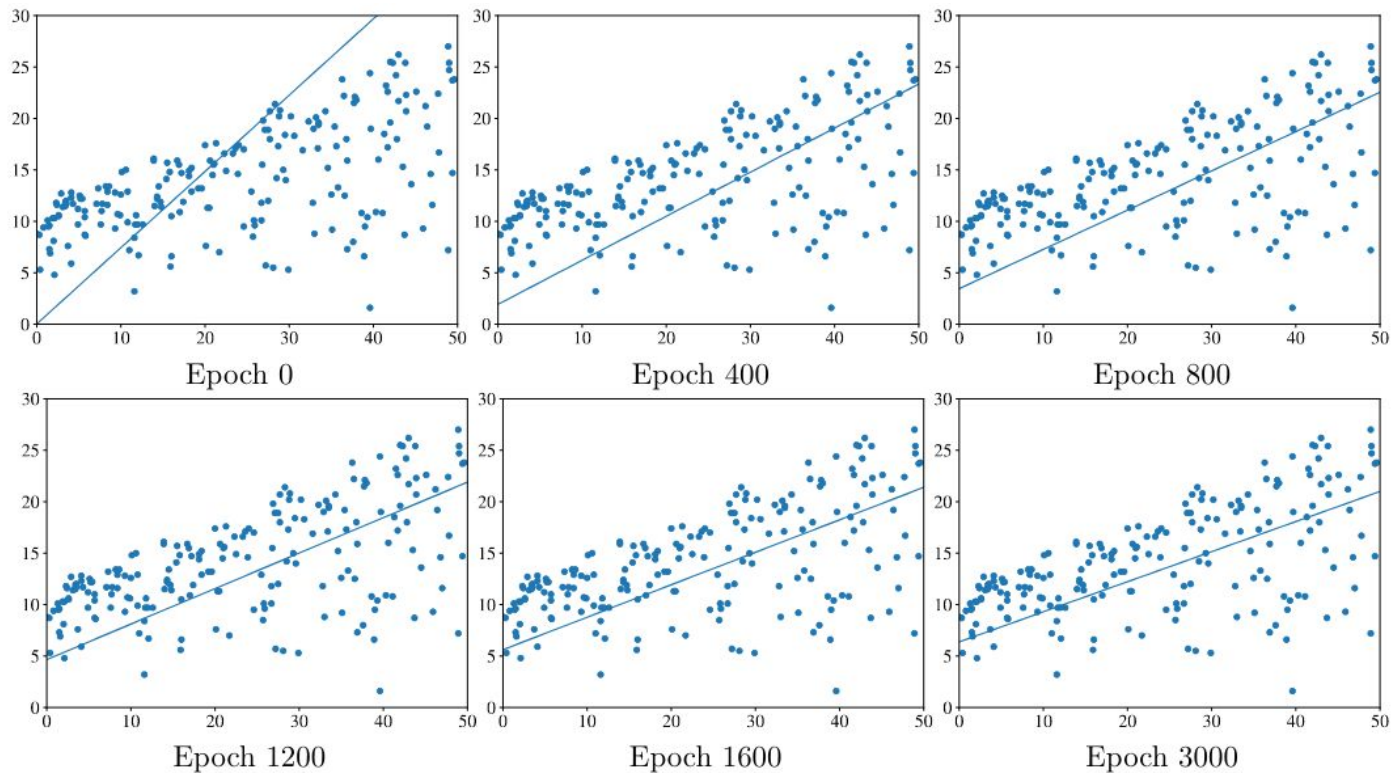


Figure 2: The evolution of the regression line through gradient descent epochs.

Descenso del gradiente

El **Descenso de Gradiente** (*Gradient Descent*) es el algoritmo de optimización por excelencia en Machine Learning. Si la Función de Costo es el mapa de una montaña, el Descenso de Gradiente es el guía que nos indica cómo bajar hasta el punto más profundo.

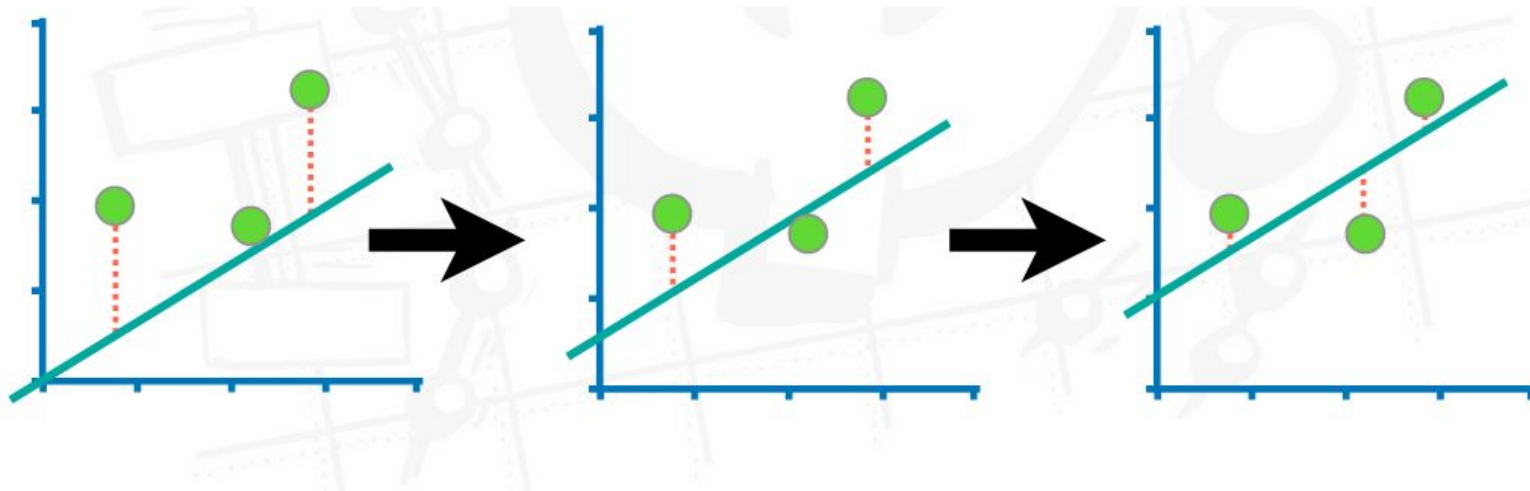
Es un algoritmo de **optimización iterativo** de primer orden que se utiliza para hallar el mínimo local (o global en funciones convexas) de una función diferenciable. Su funcionamiento se basa en actualizar los parámetros del modelo en la **dirección opuesta al gradiente** de la función de costo respecto a dichos parámetros.

En términos simples: Es el proceso de ajustar los pesos (w) y el sesgo (b) repetidamente para que el error del modelo sea lo más pequeño posible.

El gradiente (∇) es un vector que contiene todas las derivadas parciales de la función de costo. Su propiedad matemática fundamental es que **apunta hacia la dirección de máximo crecimiento**.

Por lo tanto, para minimizar el error, el algoritmo debe moverse exactamente en la dirección contraria (el **gradiente negativo**).

Descenso por el gradiente



Arrancamos con un valor aleatorio

Luego, mejoramos ese valor, una vez en cada paso

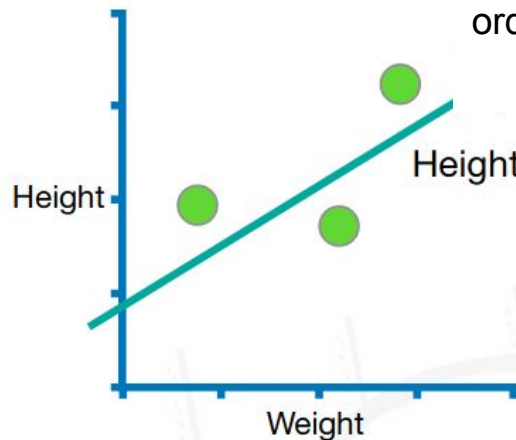
Hasta que alcanzamos una solución óptima

Fuente: *Stochastic gradient descent - study guide - Josh Hammer*

Descenso por el gradiente

Lo esencial es tener una idea intuitiva del descenso por el gradiente

El descenso por el gradiente se usa para optimizar parámetros. En este caso, queremos optimizar el valor de la ordenada y pendiente.



$$\text{Height} = \text{intercept} + 0.64 \times \text{Weight}$$

Por ahora vamos a tomar un valor inicial de 0.64 para la pendiente

Utilizamos la función de pérdida para evaluar los posibles candidatos a “mejor” parámetro.

Descenso por el gradiente

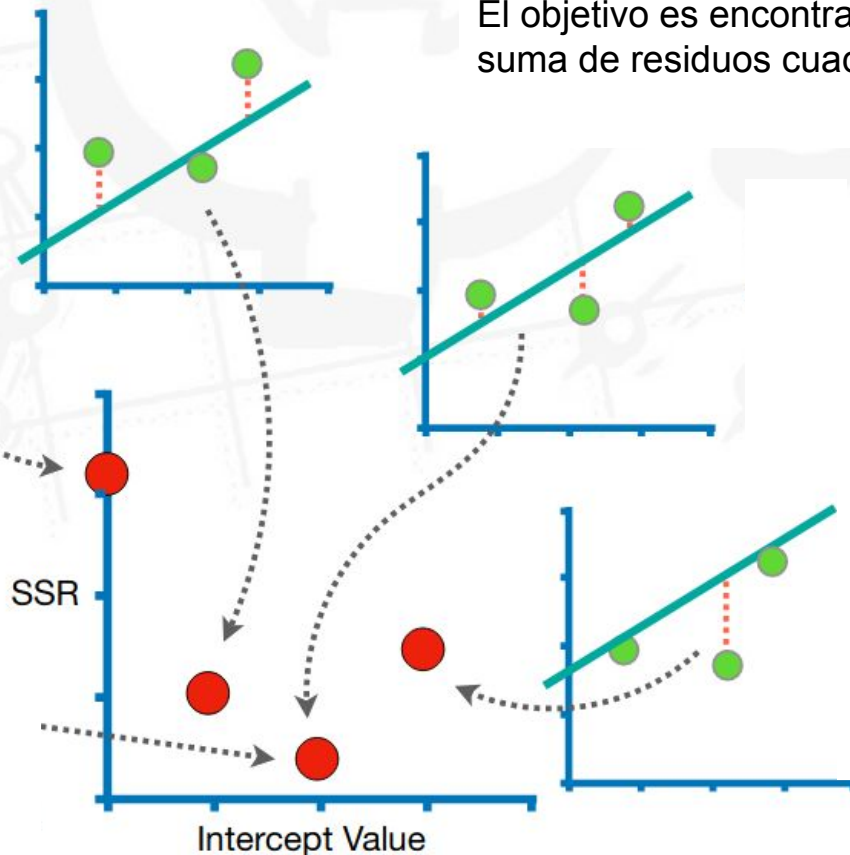
Diferentes valores de ordenadas resultan en diferentes sumas de residuales al cuadrado

$$1.1^2 + 0.4^2 + 1.3^2 = 3.1$$

Este pareciera ser el mínimo de la suma de residuos cuadrados, pero otro podría ser posible.

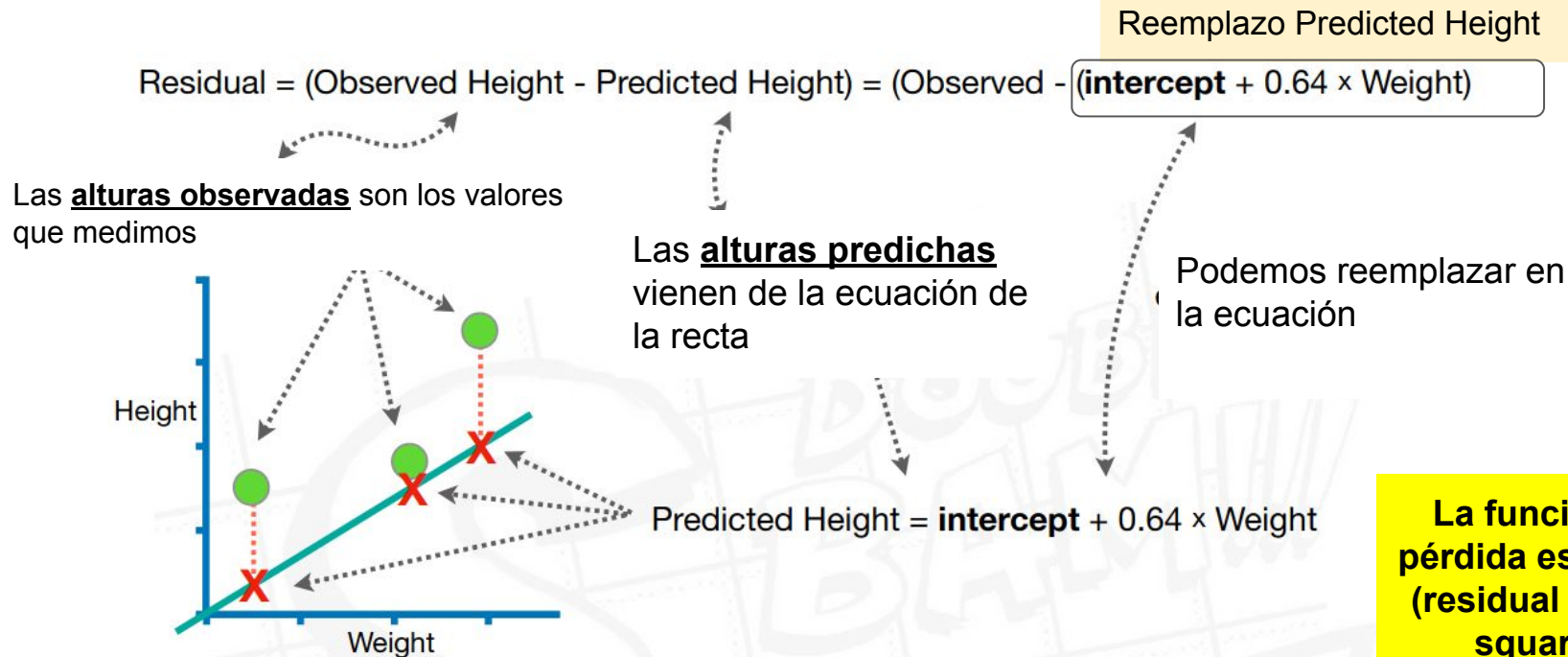
El objetivo es encontrar el **mínimo** de la suma de residuos cuadrados

El descenso por el gradiente resuelve este problema testeando relativamente pocos valores por fuera de la solución óptima e incrementando la cantidad de valores cercanos a la solución óptima.



Descenso por el gradiente

Los Residuales son la diferencia entre los valores observados y los predichos



Fuente: Stochastic gradient descent - study guide - Josh Hammer

Función de pérdida

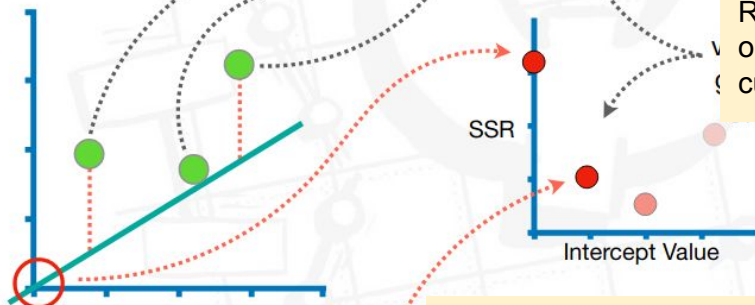
$$\text{Sum of Squared Residuals (SSR)} = (\text{Height} - (\text{intercept} + 0.64 \times \text{Weight}))^2$$

Hay un término en la suma de cada punto observado

$$+ (\text{Height} - (\text{intercept} + 0.64 \times \text{Weight}))^2$$
$$+ (\text{Height} - (\text{intercept} + 0.64 \times \text{Weight}))^2$$

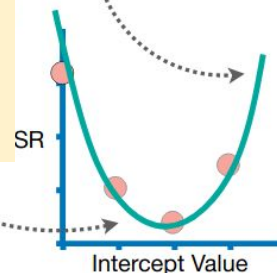
The equation for the **SSR**...

Reemplazando diferentes valores de la ordenada nos da diferentes sumas cuadradas de residuales



El objetivo es encontrar el valor de ordenada que resulta en el mínimo SSR y que corresponde al punto más bajo de la curva

corresponde a esta curva



- La gran diferencia está en que los parámetros los vamos a “aprender” mediante el descenso por el gradiente.
- Esta solución no es exacta.
- El descenso por el gradiente es la base para otros modelos más complejos.
- El objetivo: minimizar la función de costo J

OLS vs Descenso del gradiente

	OLS	Descenso del gradiente
<i>Solución</i>	Encuentra una solución exacta para los parámetros.	Encuentra una solución aproximada .
<i>Método de Cálculo</i>	Resuelve directamente mediante una fórmula cerrada.	Ajusta los parámetros iterativamente para minimizar el error.
<i>Complejidad Computacional</i>	Menor para datasets pequeños o medianos.	Menor para datasets grandes o complejos.
<i>Tiempo</i>	Puede ser más rápido en datasets pequeños.	Puede ser más rápido en datasets muy grandes o con muchas variables.
<i>Tasa de Aprendizaje</i>	No se requiere ajustar una tasa de aprendizaje.	La elección de la tasa de aprendizaje es crítica para la convergencia.

OLS vs Descenso del gradiente

	OLS	Descenso del gradiente
<i>Convergencia</i>	Converge en un solo paso	Converge lentamente en múltiples pasos, dependiendo de la tasa de aprendizaje.
<i>Dependencia del Modelo</i>	Utilizado mayormente en modelos lineales simples.	Base para algoritmos más complejos como redes neuronales.
<i>Robustez ante multicolinealidad</i>	Menos efectivo con multicolinealidad entre las variables.	Puede ser más flexible con multicolinealidad, pero depende del algoritmo.
<i>Necesidad de memoria</i>	Mayor, ya que requiere invertir matrices (costoso para grandes datasets).	Menor, ya que no requiere inversión de matrices.
<i>Uso en la práctica</i>	Más común en situaciones donde el tamaño de los datos es manejable.	Muy común en algoritmos de aprendizaje automático, sobre todo en problemas con grandes volúmenes de datos.

El método de Mínimos Cuadrados Ordinarios (OLS) nos da una **solución exacta** a problemas de regresión lineal. Sin embargo, **no siempre es factible calcular esa solución exacta** debido a los **altos costos computacionales** que puede implicar, especialmente cuando se trabaja con grandes cantidades de datos o modelos con muchas variables.

En estos casos, **el descenso por gradiente** se presenta como una alternativa eficiente para obtener **una solución aproximada** que minimice la función de pérdida y encuentre los mejores parámetros. Este método no calcula la solución directamente, sino que ajusta los parámetros iterativamente, lo que lo hace más escalable para problemas grandes.

El descenso por gradiente “aprende” los parámetros de la recta de regresión lineal (en el caso de la regresión), pero nosotros solo controlamos los hiperparámetros. Estos incluyen el valor inicial de los parámetros, la cantidad de iteraciones, la tasa de aprendizaje, y en algunos casos, cómo esta tasa se ajusta dinámicamente (por ejemplo, con el decaimiento de la tasa de aprendizaje).

Este método es crucial porque permite optimizar parámetros en modelos más complejos como las redes neuronales profundas (deep learning), donde las soluciones exactas como OLS son inviables debido a la alta dimensionalidad y complejidad de los modelos.

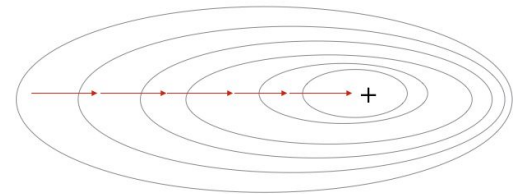
Variantes del descenso del gradiente

Batch Gradient Descent: Procesar todo el dataset a la vez. La velocidad es baja, y se requiere mucha memoria.

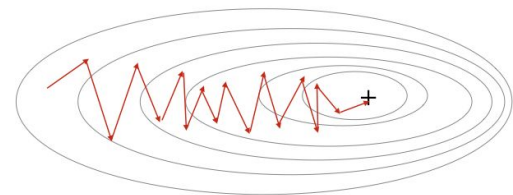
Stochastic Gradient Descent (SGD): Actualizar parámetros tras cada ejemplo. Es vital para entender cómo entrenar con datos masivos. La velocidad es muy rápida y la memoria requerida es poca.

Mini-batch Gradient Descent: El punto medio que se usa en redes neuronales profundas. La velocidad es rápida (optimizada por GPU) y la memoria requerida es moderada.

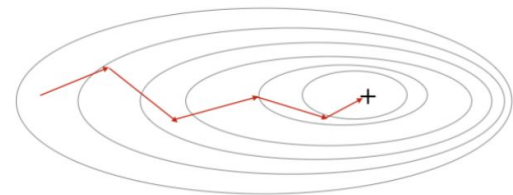
Gradient Descent



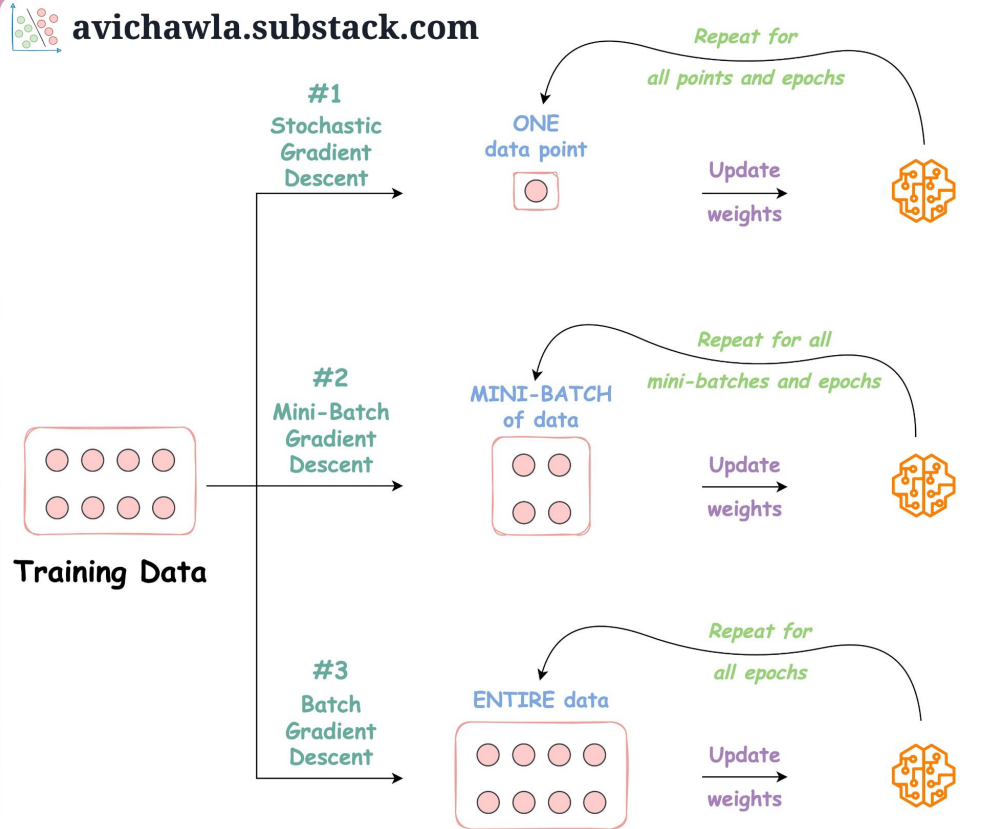
Stochastic Gradient Descent



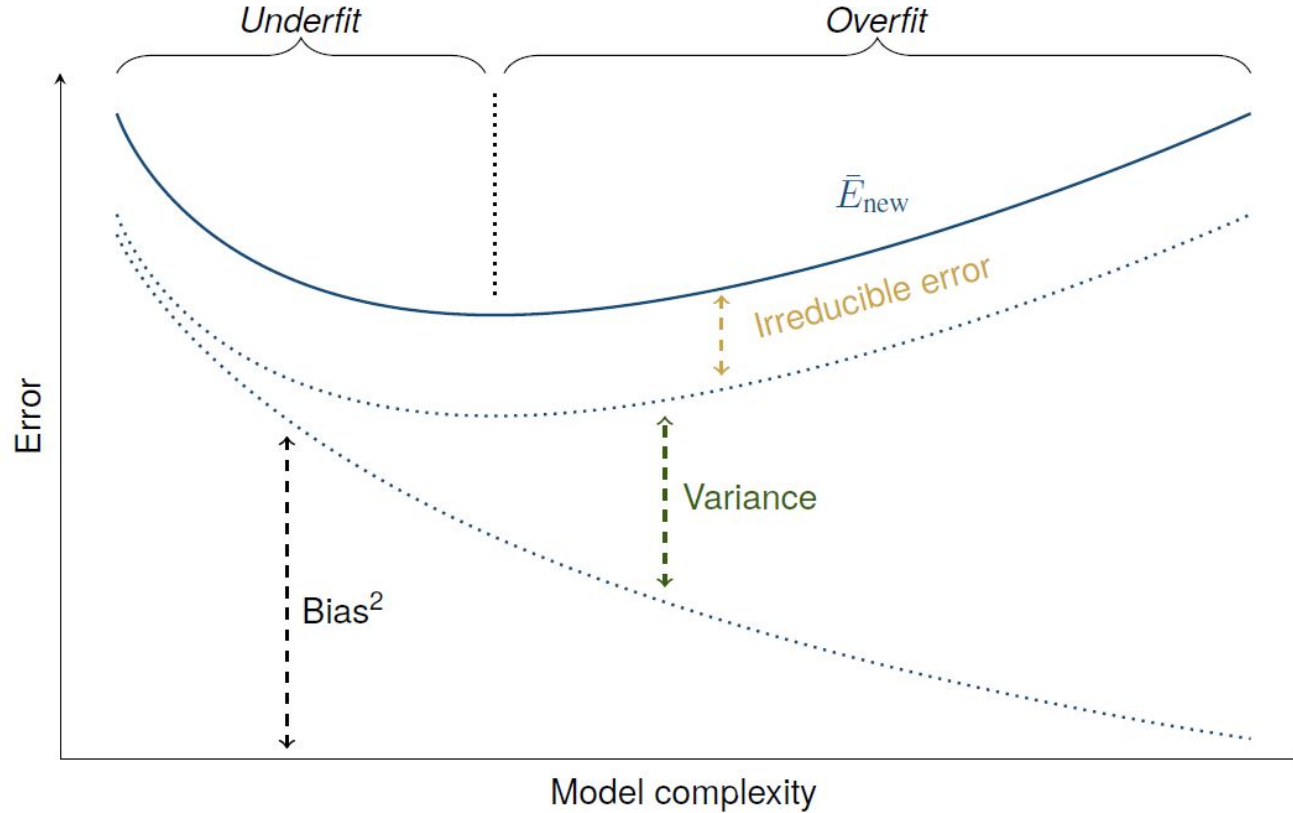
Mini-Batch Gradient Descent



Variantes del descenso del gradiente



Bias variance tradeoff



Bias variance tradeoff

How far is a
model from the
ground truth



$$E[(\hat{y} - y)^2]$$

MSE

=

How far is a
model from the
«average model»



$$E[(\hat{y} - E[\hat{y}])^2]$$

Variance

+

How far is the
«average model»
from the ground truth



$$(E[\hat{y}] - y)^2$$

*Bias*²

Regularización

La **regularización** es una técnica fundamental en el aprendizaje automático que consiste en introducir una penalización adicional a la complejidad del modelo durante su entrenamiento.

El Propósito: Cuando un modelo es demasiado complejo, tiende a "memorizar" el ruido y los detalles irrelevantes de los datos de entrenamiento en lugar de aprender el patrón general. La regularización actúa como un freno, forzando al modelo a mantenerse simple para que pueda generalizar mejor ante datos nuevos que nunca ha visto.

Matemáticamente, la regularización modifica la **función de pérdida** (como el Error Cuadrático Medio) añadiendo un término de penalización que depende de la magnitud de los pesos o coeficientes del modelo (w).

- **Sin regularización:** El modelo solo busca minimizar el error (qué tan cerca está de los puntos).
- **Con regularización:** El modelo debe minimizar el error **Y** mantener los coeficientes pequeños.

La regularización aplica el principio de la Navaja de Ockham al aprendizaje automático: **"En igualdad de condiciones, la explicación más sencilla suele ser la correcta"**. Al penalizar los coeficientes grandes, evitamos que el modelo dependa excesivamente de una sola característica o que cree curvas extremadamente sinuosas para pasar por todos los puntos de entrenamiento.

Regularización L2 o Ridge

La **Regularización L2**, comúnmente conocida como **Regresión Ridge**, es una técnica de optimización que busca mejorar la capacidad de generalización de un modelo de regresión lineal.

L2 Regularization

$$\text{Cost} = \sum_{i=0}^N (y_i - \sum_{j=0}^M x_{ij} W_j)^2 + \lambda \sum_{j=0}^M W_j^2$$

Término de
regularización

Si $\lambda = 0$: La penalización se anula y obtenemos una regresión lineal estándar por Mínimos Cuadrados Ordinarios.

Si λ es muy alto: La penalización domina y los coeficientes tienden a cero, lo que puede simplificar demasiado el modelo (riesgo de *underfitting*).

Objetivo: Encontrar un valor de equilibrado λ que reduzca la varianza sin aumentar demasiado el sesgo.

Impacto de λ en Ridge

Valor de λ	Complejidad del Modelo	Sesgo (Bias)	Varianza	Efecto en Variables
$\lambda = 0$	Máxima (OLS)	Bajo	Alta	Los coeficientes son calculados por Mínimos Cuadrados Ordinarios sin restricciones.
λ Óptimo	Balanceada	Moderado	Moderada	Los coeficientes se reducen (<i>shrinkage</i>), penalizando a los más grandes para estabilizar el modelo.
$\lambda \rightarrow \infty$	Mínima	Muy Alto	Muy Baja	Los coeficientes tienden a cero pero nunca son exactamente cero. El modelo se vuelve casi constante.

Regularización L1 o Lasso

La **Regularización L1**, conocida como **Lasso** (*Least Absolute Shrinkage and Selection Operator*), es una técnica que, al igual que Ridge, busca evitar el sobreajuste, pero con una propiedad única: tiene la capacidad de realizar una **selección automática de variables**.

L1 Regularization

$$\text{Cost} = \sum_{i=0}^N (y_i - \sum_{j=0}^M x_{ij} W_j)^2 + \lambda \sum_{j=0}^M |W_j|$$

Término de
regularización

Si $\lambda = 0$: La penalización se anula y obtenemos una regresión lineal estándar por Mínimos Cuadrados Ordinarios.

Si λ es muy alto: A medida que aumenta λ , la importancia de mantener los coeficientes pequeños crece hasta superar la importancia de reducir el error de predicción. A diferencia de Ridge (que solo achica los pesos), Lasso te permite **realizar una selección de variables**.

Impacto de λ en Lasso

Valor de λ	Complejidad del Modelo	Sesgo (Bias)	Varianza	Efecto en Variables
$\lambda = 0$	Máxima (OLS)	Bajo	Alta	Todas las variables se mantienen con pesos grandes.
λ Óptimo	Balanceada	Moderado	Moderada	Se mantienen sólo las variables importantes (Sparsity).
$\lambda \rightarrow \infty$	Mínima	Muy Alto	Muy Baja	Todas las variables se eliminan (pesos = 0).

Quando usar L1 o L2

Usar Ridge (L2) cuando:

- **Existen muchas variables correlacionadas:** Es el mejor para manejar la **multicolinealidad**. Si tenés variables que se mueven juntas (como la temperatura y la humedad en datos ambientales), Ridge distribuirá los pesos entre ambas en lugar de elegir una al azar.
- **Todas las variables son potencialmente importantes:** Si se sospecha que cada característica de tu dataset aporta al menos un poco de información para la predicción, Ridge es ideal porque mantiene a todas en el modelo.
- **Estabilidad:** Ridge es menos sensible a pequeños cambios en los datos de entrenamiento que Lasso.

Usar Lasso (L1) cuando:

- **Se busca un modelo interpretable:** Si necesitás explicarle a alguien (o a tus alumnos) exactamente qué variables son las que afectan el resultado, Lasso es mejor porque elimina el "ruido" y deja solo las más importantes.
- **Existe un dataset con muchísimas variables (High-Dimensionality):** Si tenés cientos de columnas pero creés que solo 10 son las que realmente importan, Lasso hará el trabajo de "limpieza" por vos automáticamente (**Feature Selection**).
- **Se busca un modelo "parsimonioso":** Es decir, un modelo que use la menor cantidad de recursos y variables posibles para obtener un buen resultado.

La **regularización Elastic Net** es una técnica de regresión lineal que combina las penalizaciones de los métodos **Ridge (L2)** y **LASSO (L1)** en una sola función de pérdida.

$$\text{Cost} = \sum_{i=0}^N (y_i - \sum_{j=0}^M x_{ij} W_j)^2 + \lambda \sum_{j=0}^M |W_j| + \lambda \sum_{j=0}^M W_j^2$$

Términos de regularización

Elastic Net se diseñó para superar las deficiencias de LASSO en escenarios específicos:

- **Variables muy correlacionadas:** Si tienes un grupo de variables altamente correlacionadas, LASSO tiende a seleccionar arbitrariamente una sola y descartar las demás. Elastic Net tiende a incluir el grupo completo (efecto de agrupación o *grouping effect*).
- **Más variables que observaciones ($n < p$):** En casos donde hay muchísimas variables pero pocos datos, LASSO sólo puede seleccionar como máximo n variables. Elastic Net no tiene esta limitación.

Es un método de **compromiso**. Se utiliza cuando se quiere un modelo que sea capaz de eliminar variables irrelevantes (como LASSO), pero que a la vez sea robusto y no descarte información importante cuando existen variables que se parecen mucho entre sí (como Ridge).

Impacto λ de en Elastic Net

Valor de λ	Complejidad del Modelo	Sesgo (Bias)	Varianza	Efecto en Variables
$\lambda = 0$	Máxima. El modelo intenta ajustarse perfectamente a cada dato.	Mínimo. No hay restricciones, el modelo es muy flexible.	Máxima. Muy sensible al ruido; riesgo alto de <i>overfitting</i> .	Los coeficientes son libres y suelen ser grandes . No se elimina ninguna variable.
λ Óptimo	Moderada. Solo mantiene la estructura necesaria para generalizar.	Bajo / Moderado. Se introduce un poco de error a propósito.	Baja. El modelo es estable y funciona bien con datos nuevos.	Los coeficientes se reducen (<i>shrinkage</i>). Algunas variables irrelevantes llegan a cero .
$\lambda \rightarrow \infty$	Mínima. El modelo se vuelve extremadamente simple.	Máxima. El modelo ignora los datos y no aprende patrones.	Mínima. No varía nada porque el modelo es básicamente una constante.	Todos los coeficientes (β) se vuelven 0. El modelo predice sólo el promedio.

Quando usar Elastic Net

1. Cuando se tienen variables altamente correlacionadas. Esta es la razón número uno.

- **El problema con LASSO (L1):** Si se tiene dos variables muy parecidas (ej. "Ingresos" y "Capacidad de consumo"), LASSO elegirá una al azar y pondrá la otra en cero. Esto puede hacer que se pierda información relevante o que el modelo sea inestable.
- **La solución de Elastic Net:** Gracias al componente Ridge (L2), Elastic Net mantiene el **"efecto de grupo"**. Si un grupo de variables están correlacionadas, tenderá a incluirlas a todas en el modelo o a quitarlas a todas juntas, en lugar de elegir una al azar.

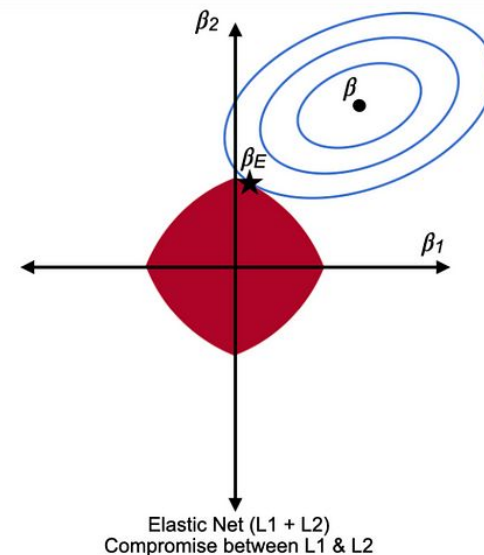
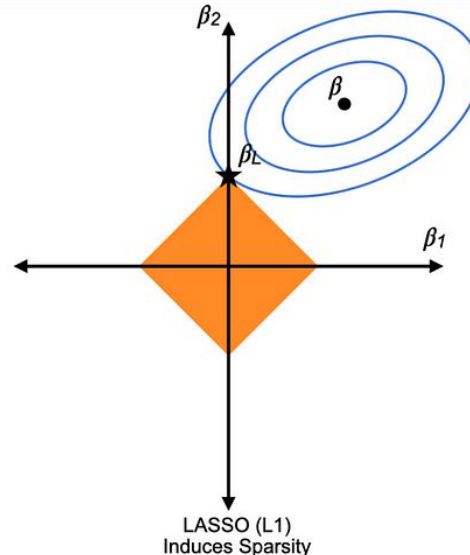
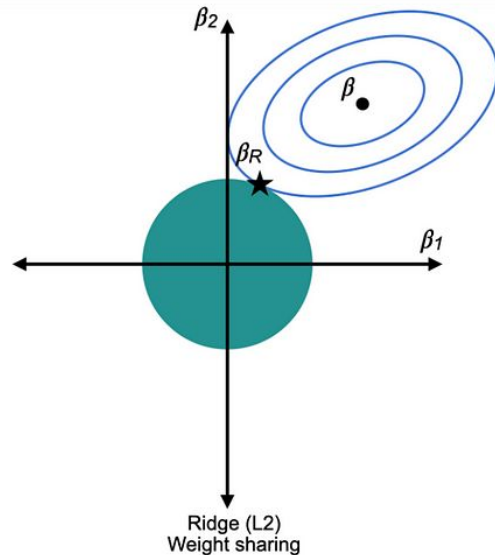
2. Cuando el número de variables (p) es mayor al número de observaciones (n)

- **El problema con LASSO (L1):** Matemáticamente, LASSO no puede seleccionar más variables que el número de filas que tienes en tu dataset. Si tienes 100 filas y 500 columnas, LASSO solo podrá elegir 100 variables como máximo.
- **La solución de Elastic Net:** Puede manejar situaciones donde $p > n$ de forma mucho más efectiva, permitiendo seleccionar un número mayor de variables si la información es relevante.

3. Cuando buscas un balance entre "Interpretabilidad" y "Precisión"

- **Ridge (L2)** es excelente para predicción porque usa toda la información, pero es **difícil de interpretar** porque mantiene todas las variables (aunque sean miles).
- **LASSO (L1)** es excelente para **interpretar** (te deja solo las variables importantes), pero puede ser demasiado agresivo y perder precisión.
- **Elastic Net** te permite "calibrar" qué tanto quieres de cada mundo.

Visualiza el equilibrio entre la función de costo y la restricción.



β RSS (Least Square) Coefficients  Contours of RSS

β_R Ridge Coefficients

β_L LASSO Coefficients

β_E Elastic Net Coefficients

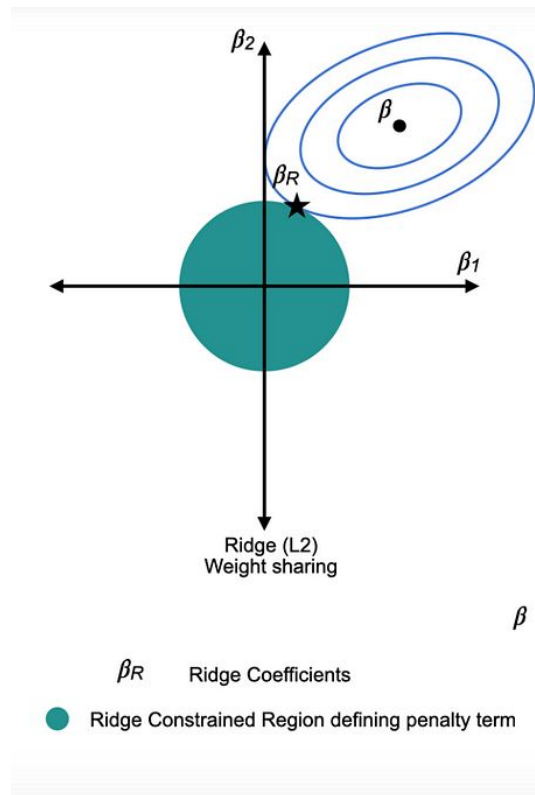
 Ridge Constrained Region defining penalty term

 LASSO constrained region defining penalty term

 Elastic Net constrained region defining penalty term

LASSO - Least Absolute Shrinkage and Selection Operator

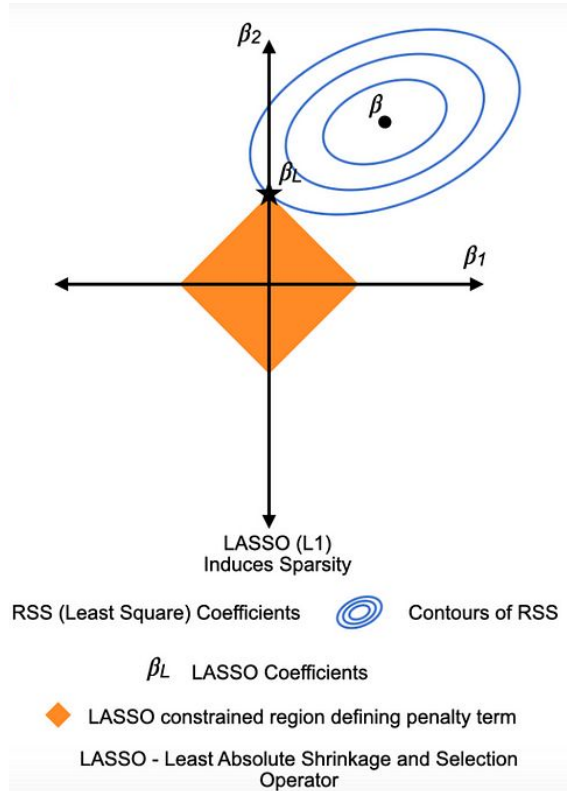
Ridge (L2) - Círculo



En la regresión Ridge, la penalización es proporcional al cuadrado de los coeficientes: $\sum \beta_j^2 \leq s$. Esto genera una restricción con forma de **círculo** (en 2D).

- **Geometría:** Como el círculo es suave y redondeado, es muy improbable que el punto de contacto con el error (la estrella β_R) ocurra exactamente sobre un eje.
- **Resultado:** Los coeficientes se reducen mucho (se acercan al cero), pero **casi nunca llegan a ser cero**. Es ideal cuando tienes muchas variables que contribuyen un poco al resultado ("Weight sharing").

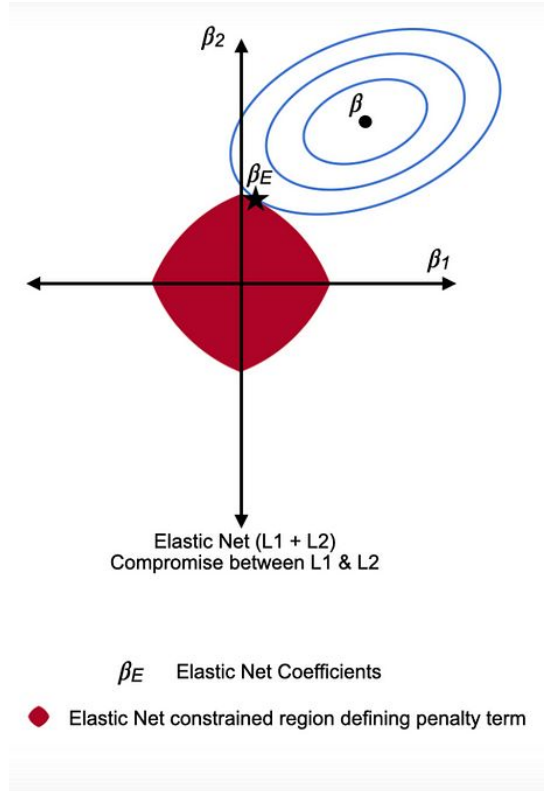
Lasso (L1) - Diamante



LASSO utiliza el valor absoluto de los coeficientes para penalizar: $\sum |\beta_j| \leq s$. Esto genera una restricción con forma de **diamante** o cuadrado rotado.

- **Geometría:** El diamante tiene **esquinas** afiladas que se encuentran justo sobre los ejes (β_1 o β_2). Los contornos del error suelen chocar primero con una de estas esquinas.
- **Resultado:** Al chocar en una esquina, uno de los coeficientes se vuelve **exactamente cero**. Por eso se dice que "induce sparsity" (dispersión): funciona como un seleccionador de variables automático, eliminando las que no sirven.

Elastic Net - Híbrido



Elastic Net es una combinación de las dos anteriores (L1 + L2). La forma de su restricción es un punto medio: un **diamante con bordes redondeados**.

- **Geometría:** Busca el equilibrio. Mantiene parte del poder de selección de LASSO (esquinas) pero con la estabilidad de Ridge (curvatura).
- **Resultado:** Es especialmente útil cuando tienes variables muy correlacionadas entre sí. Mientras que LASSO podría elegir una al azar y descartar las otras, Elastic Net tiende a mantener grupos de variables relacionadas.

Resumen de diferencias

Característica	Ridge (L2)	Lasso (L2)	Elastic Net
Forma geométrica	Círculo	Diamante	Diamante redondeado
Efecto Principal	Reduce coeficientes	Selecciona variables	Equilibrio / Grupos
¿Llega a cero?	Casi nunca	Sí (frecuente)	Sí

Regularization in Machine Learning

Ridge

$$Loss = \sum (y_i - \hat{y}_i)^2 + \lambda \beta_j^2$$

Adds L2 penalty term, shrinks coefficients toward zero but not exactly zero

Lasso

$$Loss = \sum (y_i - \hat{y}_i)^2 + \lambda |\beta_j|$$

Adds L1 penalty term, shrinks some coefficients to exactly zero

Elastic Net

$$Loss = \sum (y_i - \hat{y}_i)^2 + \lambda_1 |\beta_j| + \lambda_2 \beta_j^2$$

Combines L1 and L2 penalty terms, balances sparsity and regularization

Cuando utilizar cada tipo de regularización

El tip de "Oro": La Regla del Pulgar

- Si se tienen **pocas variables** y mucha confianza en que todas sirven: **Ridge**.
- Si se tienen **muchas variables** y se quiere simplificar: **Lasso**.
- Si se está en una situación intermedia donde hay muchas variables correlacionadas pero también se quiere eliminar las que no sirven: **Elastic Net** (que es una combinación de ambas).

¿Cómo decidir científicamente?

En la práctica, lo que se suele hacer es probar ambos mediante **Validación Cruzada (Cross-Validation)**:

- Entrenar un modelo Ridge y uno Lasso.
- Comparar el error (MSE o R^2) en el conjunto de testeo.
- El que arroje un error menor con el λ óptimo es el que se debería elegir.



OLS

$$\min_{\beta} \sum_{\beta}^{\beta} (y_i - \hat{y}_i)^2$$

Lasso

$$\min_{\beta} \sum_{\beta}^{\beta} (y - \hat{y}_i)^2 + \lambda |\beta_j|$$

Ridge

$$\min_{\beta} \sum_{\beta}^{\beta} (y_i - \hat{y}_i)^2 + \lambda_1 \beta_j^2$$

ElasticNet

$$\min_{\beta} \sum_{\beta}^{\beta} (y - \hat{y}_i)^2 + \frac{\lambda_1 |\beta_j|}{2} + \lambda_2 \beta_j^2$$

Aplicaciones espaciales



Contents lists available at [ScienceDirect](https://www.sciencedirect.com)

Epidemics

journal homepage: www.elsevier.com/locate/epidemics



Spatial analysis made easy with linear regression and kernels

Philip Milton^{a,*}, Helen Coupland^a, Emanuele Giorgi^b, Samir Bhatt^a

^a MRC Centre for Outbreak Analysis and Modelling, Department of Infectious Disease Epidemiology, Imperial College London, London, UK

^b CHICAS, Lancaster Medical School, Lancaster University, Lancaster, UK



ARTICLE INFO

Keywords:

Regression
Random Fourier features
Kernel methods
Kernel approximation

ABSTRACT

Kernel methods are a popular technique for extending linear models to handle non-linear spatial problems via a mapping to an implicit, high-dimensional feature space. While kernel methods are computationally cheaper than an explicit feature mapping, they are still subject to cubic cost on the number of points. Given only a few thousand locations, this computational cost rapidly outstrips the currently available computational power. This paper aims to provide an overview of kernel methods from first-principles (with a focus on ridge regression) and progress to a review of random Fourier features (RFF), a method that enables the scaling of kernel methods to big datasets. We show how the RFF method is capable of approximating the full kernel matrix, providing a significant computational speed-up for a negligible cost to accuracy and can be incorporated into many existing spatial methods using only a few lines of code. We give an example of the implementation of RFFs on a simulated spatial data set to illustrate these properties. Lastly, we summarise the main issues with RFFs and highlight some of the advanced techniques aimed at alleviating them. At each stage, the associated R code is provided.

Estimation of Spatial and Temporal Autoregressive Effects using LASSO – An Example of Hourly Particulate Matter Concentrations

Elkanah Nyabuto*

University of Glasgow, UK

Philipp Otto[†]

University of Glasgow, UK

Yarema Okhrin[‡]

University of Augsburg, Germany

Práctica en Python

<https://towardsdatascience.com/least-squares-regression-explained-a-visual-guide-with-code-examples-for-beginners-2e5ad011eae4/>

<https://towardsdatascience.com/lasso-and-elastic-net-regressions-explained-a-visual-guide-with-code-examples-5fecf3e1432f/>

https://rpubs.com/facundol/trabajo_final_especializacion_data_mining

<https://devopedia.org/bias-variance-trade-off>

<https://www.datacamp.com/tutorial/loss-function-in-machine-learning>